# Module Interface Specification for FitFolio

Team #1, Burn n' Earn
Davis Lenover
Preston Grewal
Angad Chhabra
Farid Bastoros
Zaid Alsaadi

November 10, 2026

# 1 Revision History

| Date | Version | Notes |
|---|---|---|
| Nov 7, 2026 | 1.0 | Davis Lenover: Revision −1. 4 Notation, and the MIS of Database, Clock, Market Providers, Control, Constants, Market, Trading, Plugins and JSON Format. |
| Nov 9, 2026 | 1.1 | Davis Lenover: the MIS of Users and Wallet, with the reason for each decision. 1 to 5: Revision History, Symbols, Introduction, Notation and Module Decomposition. |
| Nov 10, 2026 | 1.2 | Davis Lenover: the reason for each decision in every other server module, Market Providers as a template module, error handlers in JSON Format, and the 401 on both Trading routes. |

# 2 Symbols, Abbreviations and Acronyms

This document uses the terms and acronyms of the SRS (SRS Section 1.4) and its symbolic constants, such as MAX_DAILY_CURRENCY (SRS Appendix C). Constants (M7) gives the code their values, which must match the SRS. The abbreviations this document uses are:

| Symbol | Description |
|---|---|
| API | Application Programming Interface |
| FR | Functional Requirement (SRS), by its printed number |
| H | Hazard (HA), such as H51 |
| HA | Hazard Analysis |
| HTTP | Hypertext Transfer Protocol |
| IANA | Internet Assigned Numbers Authority, which names time zones |
| ID | Identifier |
| JSON | JavaScript Object Notation |
| LC | Likely Change (SRS), such as LC3 |
| M | Module (MG), such as M8 |
| MG | Module Guide |
| MIS | Module Interface Specification |
| NaN | Not a Number, a Decimal value that is not a number |
| NFR | Nonfunctional Requirement (SRS) |
| SC | System Constraint (SRS), such as SC4 |
| SEC | Security Requirement (SRS and HA) |
| SRS | Software Requirements Specification |
| URL | Uniform Resource Locator |
| USD | United States dollar |
| UTC | Coordinated Universal Time |
| UUID | Universally Unique Identifier |

# Contents


**1 Revision History** i

**2 Symbols, Abbreviations and Acronyms** ii

**3 Introduction** 1

**4 Notation** 2

**5 Module Decomposition** 3

**6 MIS of Database** 4
- 6.1 Module . . . 4
- 6.2 Uses . . . 4
- 6.3 Syntax . . . 4
  - 6.3.1 Exported Types . . . 4
  - 6.3.2 Exported Constants . . . 5
  - 6.3.3 Exported Access Programs . . . 5
- 6.4 Semantics . . . 5
  - 6.4.1 State Variables . . . 5
  - 6.4.2 Environment Variables . . . 5
  - 6.4.3 Assumptions . . . 6
  - 6.4.4 Access Routine Semantics . . . 6
  - 6.4.5 Local Functions . . . 6

**7 MIS of Clock** 7
- 7.1 Module . . . 7
- 7.2 Uses . . . 7
- 7.3 Syntax . . . 7
  - 7.3.1 Exported Types . . . 7
  - 7.3.2 Exported Constants . . . 7
  - 7.3.3 Exported Access Programs . . . 7
- 7.4 Semantics . . . 7
  - 7.4.1 State Variables . . . 7
  - 7.4.2 Environment Variables . . . 7
  - 7.4.3 Assumptions . . . 7
  - 7.4.4 Access Routine Semantics . . . 8
  - 7.4.5 Local Functions . . . 8

**8 MIS of Market Providers** 9
- 8.1 Template Module . . . 9
- 8.2 Uses . . . 9
- 8.3 Syntax . . . 9

8.3.1 Exported Types . . . 9
8.3.2 Exported Constants . . . 9
8.3.3 Exported Access Programs . . . 10
8.4 Semantics . . . 10
8.4.1 State Variables . . . 10
8.4.2 Environment Variables . . . 10
8.4.3 Assumptions . . . 10
8.4.4 Access Routine Semantics . . . 10
8.4.5 Local Functions . . . 12

**9 MIS of Control** **13**
9.1 Module . . . 13
9.2 Uses . . . 13
9.3 Syntax . . . 13
9.3.1 Exported Types . . . 13
9.3.2 Exported Constants . . . 13
9.3.3 Exported Access Programs . . . 13
9.4 Semantics . . . 13
9.4.1 State Variables . . . 13
9.4.2 Environment Variables . . . 14
9.4.3 Assumptions . . . 14
9.4.4 Access Routine Semantics . . . 14
9.4.5 Local Functions . . . 15

**10 MIS of Constants** **16**
10.1 Module . . . 16
10.2 Uses . . . 16
10.3 Syntax . . . 16
10.3.1 Exported Types . . . 16
10.3.2 Exported Constants . . . 16
10.3.3 Exported Access Programs . . . 17
10.4 Semantics . . . 17
10.4.1 State Variables . . . 17
10.4.2 Environment Variables . . . 17
10.4.3 Assumptions . . . 17
10.4.4 Access Routine Semantics . . . 17
10.4.5 Local Functions . . . 17

**11 MIS of Users** **18**
11.1 Module . . . 18
11.2 Uses . . . 18
11.3 Syntax . . . 18
11.3.1 Exported Types . . . 18

11.3.2 Exported Constants . . . 18
11.3.3 Exported Access Programs . . . 18
11.4 Semantics . . . 19
11.4.1 State Variables . . . 19
11.4.2 Environment Variables . . . 19
11.4.3 Assumptions . . . 19
11.4.4 Access Routine Semantics . . . 19
11.4.5 Local Functions . . . 19

**12 MIS of Wallet** **20**
12.1 Module . . . 20
12.2 Uses . . . 20
12.3 Syntax . . . 20
12.3.1 Exported Types . . . 20
12.3.2 Exported Constants . . . 20
12.3.3 Exported Access Programs . . . 21
12.4 Semantics . . . 21
12.4.1 State Variables . . . 21
12.4.2 Environment Variables . . . 21
12.4.3 Assumptions . . . 22
12.4.4 Access Routine Semantics . . . 22
12.4.5 Local Functions . . . 23

**13 MIS of Market** **24**
13.1 Module . . . 24
13.2 Uses . . . 24
13.3 Syntax . . . 24
13.3.1 Exported Types . . . 24
13.3.2 Exported Constants . . . 24
13.3.3 Exported Access Programs . . . 25
13.4 Semantics . . . 25
13.4.1 State Variables . . . 25
13.4.2 Environment Variables . . . 26
13.4.3 Assumptions . . . 26
13.4.4 Access Routine Semantics . . . 26
13.4.5 Local Functions . . . 27

**14 MIS of Trading** **30**
14.1 Module . . . 30
14.2 Uses . . . 30
14.3 Syntax . . . 30
14.3.1 Exported Types . . . 30
14.3.2 Exported Constants . . . 30

14.3.3 Exported Access Programs . . . 31
14.4 Semantics . . . 31
14.4.1 State Variables . . . 31
14.4.2 Environment Variables . . . 31
14.4.3 Assumptions . . . 32
14.4.4 Access Routine Semantics . . . 32
14.4.5 Local Functions . . . 32

**15 MIS of Plugins** **35**
15.1 Module . . . 35
15.2 Uses . . . 35
15.3 Syntax . . . 35
15.3.1 Exported Types . . . 35
15.3.2 Exported Constants . . . 35
15.3.3 Exported Access Programs . . . 36
15.4 Semantics . . . 36
15.4.1 State Variables . . . 36
15.4.2 Environment Variables . . . 36
15.4.3 Assumptions . . . 36
15.4.4 Access Routine Semantics . . . 36
15.4.5 Local Functions . . . 36

**16 MIS of JSON Format** **37**
16.1 Module . . . 37
16.2 Uses . . . 37
16.3 Syntax . . . 37
16.3.1 Exported Types . . . 37
16.3.2 Exported Constants . . . 37
16.3.3 Exported Access Programs . . . 38
16.4 Semantics . . . 38
16.4.1 State Variables . . . 38
16.4.2 Environment Variables . . . 38
16.4.3 Assumptions . . . 38
16.4.4 Access Routine Semantics . . . 38
16.4.5 Local Functions . . . 39

**17 Appendix — Generative AI Use Disclosure** **41**
17.1 AI Tools Used . . . 41
17.2 How and Where Used . . . 41
17.3 Verification of AI-Generated Content . . . 41

**18 Appendix — Reflection** **42**

# 3 Introduction

This document is the Module Interface Specification (MIS) for FitFolio, a phone app where physical activity earns virtual currency that the user invests in a simulated stock market. FitFolio is two programs: a server, written in Python with FastAPI and backed by PostgreSQL, and a phone app, written in React Native. The Development Plan (Section 10) gives the reason for each of these choices.

The MIS gives each module's interface: the types, constants and access programs it exports, and what each access program does. The Module Guide (MG) says what each module hides and why the system is split this way. The Software Requirements Specification (SRS) and the Hazard Analysis (HA) say what the system must do. Next to each decision, a module's section gives its reason: the requirement or hazard it serves, or the design pattern it uses. A later change can then be checked against what the decision was for.

This revision specifies the modules of the proof of concept. Modules marked † in Table 1 are built after it and are specified at Revision 0, so that every section describes an interface the code really has.

The full documentation and implementation are in the project repository: https://github.com/davislenover/BurnNEarn.

# 4 Notation

The structure of the MIS for modules comes from Hoffman and Strooper (1995), with the addition that template modules have been adapted from Ghezzi et al. (2003): a template module exports a type whose values are made with new, each with its own state. The mathematical notation comes from Chapter 3 of Hoffman and Strooper (1995). For instance, the symbol := is used for a multiple assignment statement and conditional rules follow the form $(c_1 \Rightarrow r_1 | c_2 \Rightarrow r_2 | ... | c_n \Rightarrow r_n)$. The table below gives the primitive data types.

| Data Type | Notation | Description |
|---|---|---|
| integer | $\mathbb{Z}$ | a number without a fractional component in (-$\infty$, $\infty$) |
| natural number | $\mathbb{N}$ | a number without a fractional component in [1, $\infty$) |
| real | $\mathbb{R}$ | any number in (-$\infty$, $\infty$) |
| boolean | $\mathbb{B}$ | a value that is either true or false |
| string | String | a sequence of characters |
| decimal | Decimal | an exact decimal number, such as a price, so money is never rounded by accident (NFR1) |
| date | Date | a calendar day, without a time of day |
| date and time | DateTime | a moment in time with its time zone, so the server, the phone and the market agree on it (FR33, SEC-24) |
| duration | Duration | a length of time, such as 15 minutes |
| identifier | ID | a UUID, which the phone can make without the database, such as a trade's request ID (FR38) |

FitFolio also uses derived types. A sequence of values of one type T is "seq of T", a set is "set of T", and a tuple, whose values may have different types, is "tuple of (name: T, ...)". A value that may be missing is "T or None". A function from T1 to T2 is T1 $\rightarrow$ T2, and a local function is given by its type signature and then its specification. "new X" or X($v$, ...) makes a value of type X. $t$.f is field f of tuple $t$. $p$.f($x$) calls access program f of $p$, and the In column leaves $p$ out. A type or a constant is defined with =, a type written {a, b} has just the values a and b, and := gives a value in the semantics. There, transition says how the state changes, output gives *out*, the value returned, and exception gives *exc*, the exception raised. When no case of *exc* holds, nothing is raised.

Types such as Session, Plugin and DomainError are exported by a module and defined in its section. ValueError is Python's, IntegrityError, NoResultFound, OperationalError and SQLAlchemyError are SQLAlchemy's, ValidationError is Pydantic's, and RequestValidationError is FastAPI's. In an access program table, In gives each input's type in order, separated by commas, and - means none. A route is written as its HTTP method and path, such as GET /v1/me/balance. Its In column lists what the phone sends, and its Exceptions column the refusals it raises, besides JSON Format's 422 for a request that doesn't fit (SEC-25).

# 5 Module Decomposition

The following table is taken directly from the MG. Each section of this document keeps its module's MG number, such as Users (M8), so a module can be traced between the two documents. Modules marked (app) are in the phone app, Constants is in both programs, and the rest are in the server.

| **Level 1** | **Level 2** |
|---|---|
| Hardware-Hiding Module | M1 Database<br>M2 Clock<br>M3 Market Providers<br>M25 Email Service†<br>M4 Health Service (app)<br>M5 Device Service (app) |
| Behaviour-Hiding Module | M6 Control<br>M7 Constants (server and app)<br>M8 Users<br>M9 Wallet<br>M10 Market<br>M11 Activity<br>M12 Trading<br>M13 Goals†<br>M14 Learning†<br>M15 Social†<br>M16 Scenarios†<br>M17 Privacy†<br>M18 Navigation (app)<br>M19 Screens (app)<br>M20 Sync Service (app)<br>M21 Reminder Service (app)† |
| Software Decision Module | M22 Plugins<br>M23 JSON Format<br>M24 API Service (app) |

Table 1: Module Hierarchy

Modules marked † are built after the proof of concept and are specified at Revision 0.

# 6 MIS of Database

## 6.1 Module

Database (M1 in the MG)

## 6.2 Uses

None

## 6.3 Syntax

### 6.3.1 Exported Types

Session: one unit of work on the database, saved by commit and dropped by rollback or close.

Base: the type every module defines its own tables on. Every timestamp is stored as a moment (timestamptz), not as a time without a zone, so a stored time and the current time compare correctly, such as for a price's age (FR33). It may come back in a different time zone than it was saved in.

A table holds one kind of thing, such as assets. It has a row for each asset and a column for each fact about it, such as its symbol.

A table can have constraints. A constraint is a rule the database itself checks every time a row is added, changed or deleted, and it refuses a change that breaks the rule. There are five kinds, shown here with examples from Market (M10):

- Not null: every row must have a value in this column. Every column in Market is not null.
- Primary key: the column that tells each row apart, so no two rows have the same value in it. A table has one. Each asset has its own asset_id.
- Foreign key: a column that must hold the primary key of a row in another table. Each price's asset_id must be the asset_id of an asset, so a price can never belong to an asset that does not exist. For the same reason, an asset that still has prices cannot be deleted.
- Unique: no two rows may have the same value in the columns it names. A table can have several. No two assets share a symbol. With more than one column, only the combination must differ: an asset has many prices, but at most one for each quoted_at (the time the price was set).
- Check: a condition every row must meet, such as a price being above 0.

Last, a table can have an index. Like the index of a book, it lets the database find rows by a column without reading every row. An index is not a rule. The table of calls to the market data provider, provider_requests, has one on requested_at, so counting the calls in the last minute stays fast. A primary key or unique comes with its own index, so it needs no second one.

Base names every primary key, foreign key, unique, check and index that has no name of its own, from its table and columns, by the rules below, instead of letting the database choose. A migration (a script that changes the tables in the database) can then change or drop one by a name known when the migration is written:

| **Kind** | **Name** | **Example** |
|---|---|---|
| Primary key | pk_*table* | pk_assets |
| Foreign key | fk_*table_first column_other table* | fk_prices_asset_id_assets |
| Unique | uq_*table_each column* | uq_prices_asset_id_quoted_at |
| Check | ck_*table_check name* | ck_prices_price_positive |
| Index | ix_*table_first column* | ix_provider_requests_requested_at |

A check is a condition, not a list of columns, so whoever writes it gives it a name, such as price_positive, and Base adds the rest. A check without a name is an error.

### 6.3.2 Exported Constants

None

### 6.3.3 Exported Access Programs

| **Name** | **In** | **Out** | **Exceptions** |
|---|---|---|---|
| SessionLocal | - | Session | - |
| get_db | - | Session | - |

## 6.4 Semantics

### 6.4.1 State Variables

engine: the connection to the database at DATABASE_URL, made once when the module is first loaded.

### 6.4.2 Environment Variables

database: the PostgreSQL server that holds every module's tables.

DATABASE_URL: String, the database to connect to, read from the environment or .env, so tests can use their own database and a deployed server's password stays out of the repository (HA H45). Without one, it is the local database that compose.yaml starts.

### 6.4.3 Assumptions

- The migrations are applied before the server starts.
- Each module owns its tables. It never reads or writes another module's tables, though it may have a foreign key into them. So a module can change its tables without changing any other, as long as it keeps the keys other modules point to, and no other module can break their rules by accident.
- A route commits at most once per request. A route is the function in a module's routes.py that answers one HTTP request, such as GET /v1/assets in Market (M10). To commit is to save a session's changes to the database for good. Code that answers no request commits once per piece of work: the price poller in Market once per poll, and the seed script (app/scripts/seed_demo.py) once per run.

The commit rule works like this. The code a route calls, in its own module or another, changes the session but never commits. A route that changes anything commits once, after every step has succeeded. If a step refuses the request or fails, the route does not commit, so nothing is saved. So everything one request changes, such as a buy's payment and its new shares, is saved together or not at all (FR37, NFR6). The poller and the seed script make their own session with SessionLocal, since no request gives them one, and the code they call never commits either.

### 6.4.4 Access Routine Semantics

SessionLocal():

- output: *out* := a new Session on engine. The caller closes it, and commits only as the Assumptions allow.
- exception: None

get_db():

- transition: when the request ends, the session is closed and anything the route did not commit is dropped.
- output: *out* := SessionLocal(), a session for one request alone. Routes ask for it by dependency injection, so a test can replace get_db with a session whose changes are undone after the test.
- exception: None. The session raises OperationalError, a kind of SQLAlchemyError, at its first use if the database cannot be reached.

### 6.4.5 Local Functions

None

# 7 MIS of Clock

## 7.1 Module

Clock (M2 in the MG)

## 7.2 Uses

None

## 7.3 Syntax

### 7.3.1 Exported Types

Clock = () → DateTime, a function that returns the current time. Routes and background work are handed a Clock instead of reading the system time (dependency injection), so a test can set the time for each rule that depends on it: market hours, when a price is delayed or too old to trade (FR33, FR39), and, once Activity (M11) is built, the user's date (SEC-24).

### 7.3.2 Exported Constants

None

### 7.3.3 Exported Access Programs

| Name | In | Out | Exceptions |
|---|---|---|---|
| system_clock | - | DateTime | - |
| get_clock | - | Clock | - |

## 7.4 Semantics

### 7.4.1 State Variables

None

### 7.4.2 Environment Variables

osTime: DateTime, the current time from the server's operating system clock.

### 7.4.3 Assumptions

The server's operating system clock has these properties:

- It is synchronized to Coordinated Universal Time (UTC) by the Network Time Protocol, and stays within 1 second of UTC. That is close enough, since every rule that depends on the time works in whole minutes or more.
- It is not monotonic. A sync can move it back slightly, so a later call can return an earlier time. That does no harm. Its times are stored and compared with other stored times and with market hours, and nothing uses it to time a wait.

#### 7.4.4 Access Routine Semantics

system_clock():

- output: *out* := osTime in UTC, to the microsecond. UTC is the same whatever zone the server is set to and has no daylight saving time, so taking a Duration from it gives the time exactly that long ago. Market (M10) turns it into New York time for market hours.
- exception: None

get_clock():

- output: *out* := system_clock. The output is the function itself, not a time. Routes ask for get_clock as a dependency and call the Clock it gives. A test replaces get_clock so it gives a Clock that always returns a fixed time.
- exception: None

#### 7.4.5 Local Functions

None

# 8 MIS of Market Providers

## 8.1 Template Module

MarketDataProvider (Market Providers, M3 in the MG), an abstract data type whose values are providers, each made with new FinnhubProvider or new FakeProvider. Prices come from a third-party provider (SC4), which may change if its request limits or cost prevent the planned use (LC3). So every provider has this one interface (the strategy pattern), and Market (M10) works the same with any of them.

## 8.2 Uses

Clock (M2)

## 8.3 Syntax

### 8.3.1 Exported Types

Quote = tuple of (symbol: String, price: Decimal, quoted_at: DateTime), a stock's latest price and when the provider set it (not when the server asked for it). Market (M10) marks a price delayed by this time, since a quote can already be old when the server gets it, and an old quote lags the real market (SEC-2).

MarketDataProvider: the abstract type of every provider, with one access program, get_quote. FinnhubProvider turns Finnhub's replies and errors into Quotes and MarketDataErrors (the adapter pattern). FakeProvider makes up prices, so the team can develop and test without a key or an open market.

ProviderSettings = tuple of (market_provider: String, finnhub_api_key: String or None), the name of the provider to use, currently "finnhub" or "fake", and the key for Finnhub.

MarketDataError, the error for no usable price, with its kinds RateLimitedError (the request limit was hit) and UnknownSymbolError (no price for the symbol). Market (M10) treats each kind differently. A RateLimitedError ends its poll, since more calls would be refused too. An UnknownSymbolError is recorded as "unknown_symbol", not "error", so a symbol with no price is told apart from a provider that fails.

### 8.3.2 Exported Constants

FINNHUB_CALLS_PER_MINUTE: $\mathbb{N} = 60$, Finnhub's free plan limit. GET /v1/market/usage of Market (M10) reports it, so the team can check the server's calls stay within it (SC4).

### 8.3.3 Exported Access Programs

| Name | In | Out | Exceptions |
|---|---|---|---|
| new FinnhubProvider | String | FinnhubProvider | - |
| new FakeProvider | Clock | FakeProvider | - |
| get_quote | String | Quote | MarketDataError, RateLimitedError, UnknownSymbolError |
| build_provider | ProviderSettings | MarketDataProvider | ValueError |
| get_provider_settings | - | ProviderSettings | ValidationError |

## 8.4 Semantics

### 8.4.1 State Variables

Each provider has its own state, which new sets up.

key: String, a FinnhubProvider's API key. It prints as stars, and it is sent only to Finnhub, in the X-Finnhub-Token request header rather than the URL, so no logged URL or error message shows it. A leaked key would let others use up the request limit for every user (HA H16). It is never sent to the phone (SEC-33).
clock: Clock and prices: String $\rightarrow$ Decimal, a FakeProvider's clock and its last price for each symbol.

### 8.4.2 Environment Variables

finnhubApi: the Finnhub quote API, whose reply has a status, a price $c$ and its time $t$.

MARKET_PROVIDER, FINNHUB_API_KEY: String, the two fields of ProviderSettings, read from the environment or the .env file. Git ignores .env, so the key stays out of the repository (SEC-33).

### 8.4.3 Assumptions

- Every provider follows MarketDataProvider: its get_quote returns a Quote or raises a MarketDataError, never another error. After a MarketDataError, Market (M10) still saves the poll's other prices, but any other error loses the whole poll.
- build_provider is only given settings from get_provider_settings, so they hold all that the named provider needs.

### 8.4.4 Access Routine Semantics

new FinnhubProvider($k$), new FakeProvider($f$):

- output: *out* := FinnhubProvider(key = $k$) or FakeProvider(clock = $f$, no prices)

- exception: None

$p$.get_quote($s$), for a FinnhubProvider $p$:

- output: *out* := (symbol: $s$, price: $c$, quoted_at: $t$ seconds after 1970-01-01 00:00 UTC), from one request to finnhubApi holding only $s$ and key (SEC-11). It times out after 10 seconds, so a Finnhub that doesn't answer gives a MarketDataError and the poll goes on to the next symbol.

- exception: *exc* := (the request fails or its reply cannot be read ⇒ MarketDataError | status = 429 ⇒ RateLimitedError | status ≠ 200, or no finite $c$ or valid $t$ ⇒ MarketDataError | $c \leq 0 \vee t = 0$ ⇒ UnknownSymbolError), in that order. A $c$ of NaN or infinity is read as a number but is no price. Finnhub answers a symbol it has no price for with status 200, $c = 0$ and $t = 0$, not with an error, and no real price is 0 or below.

$p$.get_quote($s$), for a FakeProvider $p$:

- transition: prices($s$) := (no price for $s$ yet ⇒ start($s$) | otherwise ⇒ step(prices($s$)))

- output: *out* := (symbol: $s$, price: prices($s$), quoted_at: clock())

- exception: None, even for a symbol no real stock has

build_provider($v$):

- output: *out* := the provider that $v$ names. That is currently new FinnhubProvider with $v$'s key for "finnhub", or new FakeProvider(system_clock) for "fake". Market (M10) gets its provider only from here (the factory pattern), so a new provider changes only this module and the limit that GET /v1/market/usage reports.

- exception: *exc* := ($v$.market_provider names no provider ⇒ ValueError). It never falls back to FakeProvider, whose made-up prices would be shown as real.

get_provider_settings():

- output: *out* := the settings in MARKET_PROVIDER and FINNHUB_API_KEY, read once, on the first call that succeeds, so every later call gets the same settings, those checked when the server started. Unset, they give "fake" and None.

- exception: *exc* := (MARKET_PROVIDER names no provider, or names one that needs a key, currently only Finnhub, and the key is unset or empty ⇒ ValidationError). Market (M10) calls it when the server starts, so bad settings, such as a missing key, stop the server then, not at its first price request.

### 8.4.5 Local Functions

start: String $\to$ Decimal, start($s$) := a price from 20 to 500 that depends only on $s$, so a symbol starts at the same price on every run and in every test
step: Decimal $\to$ Decimal, step($x$) := $x$ moved up or down at random by at most 0.5%, to four decimals, so a price moves a little on each call and never reaches 0, which Market (M10) refuses

# 9 MIS of Control

## 9.1 Module

Control (M6 in the MG)

## 9.2 Uses

Database (M1), Users (M8), Wallet (M9), Market (M10), Trading (M12), Plugins (M22), JSON Format (M23)

The MG also lists Activity (M11) and Goals (M13) to Privacy (M17). Each is added to PLUGINS once it is built.

## 9.3 Syntax

### 9.3.1 Exported Types

None

### 9.3.2 Exported Constants

PLUGINS: seq of Plugin = ⟨PLUGIN of Users (M8), PLUGIN of Wallet (M9), PLUGIN of Market (M10), PLUGIN of Trading (M12)⟩, the PLUGIN of each module in the server, in the order their background work starts. Only this list, in app/registry.py, names the modules (the plug-in pattern), so adding a module never changes create_app. Listing a module here is also how database migrations find its tables.

### 9.3.3 Exported Access Programs

| Name | In | Out | Exceptions |
|---|---|---|---|
| create_app | seq of Plugin | App | - |
| GET /v1/health | - | tuple of (status: String, database: String, modules: String → ModuleHealth) | - |

## 9.4 Semantics

### 9.4.1 State Variables

app: App, the running server, built by create_app(PLUGINS) when `uvicorn app.main:app` starts the server.

### 9.4.2 Environment Variables

None

### 9.4.3 Assumptions

No two plug-ins in PLUGINS share a name, since GET /v1/health shows each module's health by its name. No two routes share a method and path. Each plug-in's lifespan and health check follow the rules in Plugins (M22).

### 9.4.4 Access Routine Semantics

create_app(*plugins*):

- output: *out* := a new App that serves GET /v1/health, which reports the health of each plug-in in *plugins*, and the routes of each plug-in in *plugins*, all under /v1, plus the API docs at /docs. The version in the path lets the phone app and the API change separately later. It answers any DomainError a route raises with handle_domain_error, and a request that doesn't fit a route's types with handle_validation_error, both from JSON Format (M23). It runs lifespan when it starts and shuts down. *plugins* is PLUGINS when none are given. A test passes its own to build a smaller app.
- exception: None

GET /v1/health() needs no signed-in user, so the team can check the server without an account.

- output: *out* := (status: $s$, database: $d$, modules: $m$), where
  - $d$ := (a query to the database succeeds ⇒ "ok" | it fails ⇒ "unavailable")
  - for each plug-in $p$ of the App with a health check, $m(p.\text{name})$ := ($p$.health() raises an error or does not return a ModuleHealth with status "ok" or "degraded" ⇒ ("degraded", "health check failed"), and the error is logged | otherwise ⇒ $p$.health()), so one broken check can't break the whole route. A plug-in without a health check is left out of $m$.
  - $s$ := ($d$ = "ok" ∧ every status in $m$ is "ok" ⇒ "ok" | otherwise ⇒ "degraded")

  The reply has HTTP status 503 when $d$ is "unavailable", since nothing works without the database. It is 200 otherwise, even when a module is degraded, since the rest of the API still works.
- exception: None

### 9.4.5 Local Functions

lifespan: Lifespan, the background work of each App that create_app builds, made from its *plugins*.

lifespan($a$):

- transition: the lifespans of *plugins* start in order when $a$ starts and stop in reverse order when $a$ shuts down, and if one fails to start, the ones already started still stop.
- exception: $exc :=$ (a lifespan raises an error $err$ while it starts $\Rightarrow err$). $a$ does not start.

# 10 MIS of Constants

## 10.1 Module

Constants (M7 in the MG)

## 10.2 Uses

None

## 10.3 Syntax

### 10.3.1 Exported Types

ActivityType = {STEPS, WHEELCHAIR_PUSHES, CYCLING, SWIMMING, OTHER_WORKOUT}, the supported activity types (SUPPORTED_ACTIVITY_TYPES in SRS Appendix C). Each value is also a String, its name in lower case, such as "steps", so the daily totals the phone sends (SEC-24) can name their type in JSON and the database can store it as text, once Activity (M11) is built.

### 10.3.2 Exported Constants

Names and values match SRS Appendix C. Each program keeps them in one file, on the server app/core/constants.py, so each value is easy to find and to trace back to the SRS. EARNING_RATES($t$) is the amount of activity type $t$ that earns one unit of virtual currency ($u_t$ in SRS Section 6.7). Decimal amounts are in virtual currency. The phone app's copy, once written, must hold the same values and ActivityType text, since the phone applies some of them itself, such as reading the last SYNC_LOOKBACK_PERIOD of activity on each sync (SEC-20).

| Name | Type | Value |
|---|---|---|
| *Activity and rewards* | | |
| EARNING_RATES | ActivityType $\to \mathbb{N}$ | 100 steps for STEPS, 100 pushes for WHEELCHAIR_PUSHES, 1 minute for each other type |
| MAX_DAILY_CURRENCY | Decimal | 300 |
| MAX_DAILY_STEPS | $\mathbb{N}$ | 150,000 |
| MAX_DAILY_PUSHES | $\mathbb{N}$ | 100,000 |
| MAX_WORKOUT_DURATION | Duration | 12 hours |
| SYNC_LOOKBACK_PERIOD | Duration | 7 days |
| *Simulated market* | | |
| QUOTE_CURRENCY | String | "USD" (US dollars) |

| Name | Type | Value |
|---|---|---|
| QUOTE_STALENESS_LIMIT | Duration | 15 minutes |
| MIN_TRADABLE_STOCKS | $\mathbb{N}$ | 50 |
| SCENARIO_START_BALANCE | Decimal | 10,000 |
| VALUE_TOLERANCE | Decimal | 0.01 |
| *Accounts and data* | | |
| MIN_USER_AGE | $\mathbb{N}$ | 18 years |
| MIN_PASSWORD_LENGTH | $\mathbb{N}$ | 8 characters |
| MAX_FAILED_SIGNINS | $\mathbb{N}$ | 5 |
| SIGNIN_LOCKOUT_PERIOD | Duration | 15 minutes |
| SESSION_TIMEOUT | Duration | 14 days |
| DATA_DELETION_PERIOD | Duration | 30 days |

#### 10.3.3 Exported Access Programs

None

## 10.4 Semantics

#### 10.4.1 State Variables

None

#### 10.4.2 Environment Variables

None

#### 10.4.3 Assumptions

None

#### 10.4.4 Access Routine Semantics

None

#### 10.4.5 Local Functions

None

# 11 MIS of Users

## 11.1 Module

Users (M8 in the MG)

## 11.2 Uses

Database (M1), Plugins (M22), JSON Format (M23). The MG also lists Clock (M2), Constants (M7) and Email Service (M25), for sign-up, sign-in, sessions and the password reset link (FR6) at Revision 0.

## 11.3 Syntax

### 11.3.1 Exported Types

CurrentUser = tuple of (user_id: ID), a read-only copy of the user a request comes from. Routes get it from get_current_user, never from the request, so the phone can't act as another user (SEC-15).

NotAuthenticatedError, a DomainError with code "not_authenticated", status 401 and the message "Sign in to continue." It refuses a request with no signed-in user (SEC-15). Status 401 is HTTP's status for that, so the phone can tell it apart from other refusals.

### 11.3.2 Exported Constants

DEMO_USER_ID: ID = 00000000-0000-4000-8000-000000000001, the proof of concept's one built-in user. It is fixed, so the seed script and get_current_user always agree on who it is.

DEFAULT_TIME_ZONE: String = "America/Toronto", the team's own, for a user made without one, such as the demo user. At Revision 0, sign-up sends the phone's time zone instead.

PLUGIN: Plugin = (name: "users", router: None, lifespan: None, health: None). Users has no routes yet. It is a plug-in anyway, so the sign-up and sign-in routes of Revision 0 (FR1, FR4) are added here without changing Control (M6).

### 11.3.3 Exported Access Programs

| Name | In | Out | Exceptions |
|---|---|---|---|
| get_current_user | - | CurrentUser | NotAuthenticatedError |
| create_user | Session, ID or None, String | ID | ValueError, IntegrityError |

## 11.4 Semantics

### 11.4.1 State Variables

users: set of tuple of (user_id: ID, time_zone: String, created_at: DateTime), every user, with each user_id once. time_zone is an IANA time zone name. Nothing reads it yet. It is kept because the server needs the user's date to refuse a daily total dated in the future (SEC-24, HA H51). The phone, not the server, puts each activity record on its day (SEC-23).

### 11.4.2 Environment Variables

DEMO_USER_ENABLED: $\mathbb{B}$, read once per process, makes every request that needs a user come from the demo user. It is off by default, so a server not set up for the demo refuses them (SEC-15).

### 11.4.3 Assumptions

- The seed script (app/scripts/seed_demo.py) makes the demo user, and its live portfolio in Wallet (M9), before a route needs them. get_current_user does not check that the user exists, so it needs no database query.

### 11.4.4 Access Routine Semantics

get_current_user():

- output: $out$ := CurrentUser(DEMO_USER_ID). Routes ask for it by dependency injection, so the phone never sends a user ID, and a test can replace it to pick the user. At Revision 0 it finds the user from the session token instead, so no route changes.
- exception: $exc$ := ($\neg$DEMO_USER_ENABLED $\Rightarrow$ NotAuthenticatedError)

create_user(session, user_id, time_zone). Unless they are given, user_id is None and time_zone is DEFAULT_TIME_ZONE. A caller can give user_id so the seed script can make the demo user with DEMO_USER_ID.

- transition: users := users $\cup\{(u$, time_zone, the database's time)$\}$, where $u$ := user_id if it is given, else a new ID. It flushes, so the database makes the new ID, but never commits, so the caller can add the user's live portfolio and commit both together.
- output: $out := u$
- exception: $exc$ := (time_zone is not an IANA time zone name $\Rightarrow$ ValueError | some user already has user_id $\Rightarrow$ IntegrityError). A bad time zone is refused before anything changes, since it would break finding the user's date (SEC-24).

### 11.4.5 Local Functions

None

# 12 MIS of Wallet

## 12.1 Module

Wallet (M9 in the MG)

## 12.2 Uses

Database (M1), Users (M8), Plugins (M22), JSON Format (M23)

## 12.3 Syntax

### 12.3.1 Exported Types

BalanceChangeReason = {activity_reward, trade_purchase}, why a balance changed. Every change keeps its reason (SEC-22), and the database refuses any other. There is no reason for a sale yet, since the proof of concept only buys. Selling (FR36) comes at Revision 0. Nothing pays a reward yet either. That reason is there for Activity (M11), and tests use it to give a portfolio currency to spend.

LockedPortfolio = tuple of (portfolio_id: ID, user_id: ID), the read-only token that lock_live_portfolio returns (a lock token). change_balance asks for one, so changing a balance without locking it first is hard to do by accident. Nothing checks where one came from.

BalanceChangeReceipt = tuple of (balance_change_id: ID, amount: Decimal, cash_balance_after: Decimal), a read-only copy of one change, so a caller can't change the record through it (SEC-22). Trading (M12) keeps the change's ID with each trade, so each trade points at the payment for it.

InsufficientFundsError, a DomainError with code "insufficient_funds", status 409 and the message "You don't have enough currency for this." It refuses a purchase that costs more than the balance (FR37). Status 409 says the request was understood but conflicts with the balance now.

### 12.3.2 Exported Constants

PLUGIN: Plugin = (name: "wallet", router: the route below, lifespan: None, health: None)

### 12.3.3 Exported Access Programs

| Name | In | Out | Exceptions |
|---|---|---|---|
| lock_live_portfolio | Session, ID | LockedPortfolio | NoResultFound |
| change_balance | Session, LockedPortfolio, Decimal, BalanceChangeReason, Date or None | BalanceChangeReceipt | ValueError, InsufficientFundsError |
| create_live_portfolio | Session, ID | ID | IntegrityError |
| delete_portfolio | Session, ID | - | - |
| live_portfolio_id | Session, ID | ID | NoResultFound |
| get_balance | Session, ID | Decimal | NoResultFound |
| activity_paid | Session, ID, Date | Decimal | ValueError |
| GET /v1/me/balance | - | tuple of (cash_balance: Money) | NotAuthenticatedError |

## 12.4 Semantics

### 12.4.1 State Variables

portfolios: set of tuple of (portfolio_id: ID, user_id: ID, cash_balance: Decimal, scenario_id: ID or None, created_at: DateTime). A portfolio is live when its scenario_id is None, and each user has at most one live portfolio. Every cash_balance is 0 or more (FR37).

changes: set of tuple of (balance_change_id: ID, portfolio_id: ID, amount: Decimal, reason: BalanceChangeReason, activity_date: Date or None, created_at: DateTime), every change to a balance, never edited. Each amount is not 0, since a change of 0 would add a record without changing the balance. It is above 0 exactly when the reason is activity_reward, so a missing minus sign can't turn a purchase into free currency. A change has an activity_date, the day of activity it pays for, exactly when the reason is activity_reward, so a day's rewards can be added up (SEC-19).

Every amount and balance has at most 4 decimals and 12 digits before the point. Four decimals match Money of JSON Format (M23) and keep rounding far below VALUE_TOLERANCE (NFR1). For every portfolio $p$, $p$.cash_balance is the sum of the amounts of its changes (SEC-22). The database does not check this sum. change_balance keeps it true, since it is the only routine that changes a cash_balance, and it adds the change in the same transaction. Deleting a portfolio deletes its changes, and other modules' rows that point at it with ON DELETE CASCADE, such as the trades and holdings of Trading (M12).

### 12.4.2 Environment Variables

None

### 12.4.3 Assumptions

- A route that changes money calls lock_live_portfolio, then change_balance, then commits once, by the commit rule of Database (M1). The lock lasts until the session's transaction ends, by a commit, a rollback or closing the session.
- Every amount, and every balance a change leads to, is below $10^{12}$, the most the columns hold. A reward is at most the daily cap, so in practice none comes near it.
- Every user a route acts for has a live portfolio. The seed script makes the demo user's.

### 12.4.4 Access Routine Semantics

lock_live_portfolio(session, user_id), with $p$ the live portfolio of user_id:

- transition: $p$ is locked until session's transaction ends. Until then, a call for $p$ from another session waits, and it then sees what the first one saved. Taking turns keeps what a request reads after the lock true until it commits, such as whether its request ID was used (FR38) or what a day's rewards have paid (SEC-19, FR18).
- output: *out* := LockedPortfolio($p$.portfolio_id, user_id)
- exception: *exc* := (user_id has no live portfolio ⇒ NoResultFound)

change_balance(session, locked, amount, reason, activity_date), with activity_date None by default and $p$ the portfolio with portfolio_id locked.portfolio_id:

- transition: $p$.cash_balance := $p$.cash_balance + amount, and changes := changes ∪{($c$, $p$.portfolio_id, amount, reason, activity_date, the database's time)} with $c$ a new ID. Adding the change with the new balance keeps the balance equal to the sum of its changes (SEC-22). It flushes, so the database makes $c$, but never commits.
- output: *out* := BalanceChangeReceipt($c$, amount, $p$.cash_balance after the change)
- exception: the first of these that holds is raised, before anything changes:
  *exc* := (amount is 0, NaN or infinite, or has over 4 decimals ⇒ ValueError
  | (reason = activity_reward) ≠ (amount > 0) ⇒ ValueError
  | (reason = activity_reward) ≠ (activity_date ≠ None) ⇒ ValueError
  | activity_date is a DateTime ⇒ ValueError
  | $p$.cash_balance + amount < 0 ⇒ InsufficientFundsError)
  A ValueError means the calling code is wrong, not a refusal the phone sees. An amount with over 4 decimals is refused, not rounded, since the database would round it quietly. A DateTime, which Python also counts as a Date, is refused, since its day depends on the time zone it is read in, so the same activity could be paid on two days (SEC-19). A balance never goes below 0, even if the caller skipped the lock, since the change and the check are one database statement.

create_live_portfolio(session, user_id):

- transition: portfolios := portfolios ∪{($q$, user_id, 0, None, the database's time)} with $q$ a new ID. It starts at 0, since it has no changes yet (SEC-22), and is kept apart from any scenario's (SEC-31). It flushes, so the database makes $q$, but never commits.
- output: *out* := $q$
- exception: *exc* := (user_id is not in Users (M8) or already has a live portfolio ⇒ IntegrityError)

delete_portfolio(session, portfolio_id):

- transition: the portfolio with portfolio_id, its changes, and other modules' rows that point at it with ON DELETE CASCADE are removed in one statement, so nothing is left pointing at it. The seed script uses it to restart the demo. It never commits.
- exception: None

live_portfolio_id(session, user_id):

- output: *out* := the portfolio_id of user_id's live portfolio
- exception: *exc* := (user_id has no live portfolio ⇒ NoResultFound)

get_balance(session, portfolio_id):

- output: *out* := the cash_balance of the portfolio with portfolio_id, including changes session has made but not committed, so a route sees its own change before it commits
- exception: *exc* := (no portfolio has portfolio_id ⇒ NoResultFound)

activity_paid(session, portfolio_id, activity_date):

- output: *out* := the sum of the amounts of the changes with portfolio_id, reason activity_reward and activity_date, or 0. Nothing calls it yet. A step sync in Activity (M11) will use it to pay only the difference, so no activity is paid twice (SEC-19) or past the daily cap (FR18).
- exception: *exc* := (activity_date is a DateTime ⇒ ValueError), as in change_balance

GET /v1/me/balance() takes session from get_db of Database (M1) and the user $u$ := get_current_user() of Users (M8).

- output: *out* := (cash_balance: $b$), where $b$ is get_balance(session, $q$) as Money, and $q$ is live_portfolio_id(session, $u$.user_id). The phone shows it, since rewards add to it (SRS use case UC2) and a purchase can't cost more (FR37). It only reads, so it doesn't lock or commit.
- exception: *exc* := the NotAuthenticatedError of get_current_user, with status 401

### 12.4.5 Local Functions

None

# 13 MIS of Market

## 13.1 Module

Market (M10 in the MG)

## 13.2 Uses

Database (M1), Clock (M2), Market Providers (M3), Constants (M7), Plugins (M22), JSON Format (M23)

## 13.3 Syntax

### 13.3.1 Exported Types

AssetInfo = tuple of (asset_id: ID, symbol: String, name: String, is_tradable: $\mathbb{B}$), a read-only copy of one asset, such as a stock. Market hands out copies, never its rows, so no other module can change an asset or a price by accident. is_tradable is false once the asset is removed from the live market. Its row stays, since its prices and the trades and holdings of Trading (M12) point at it.

CurrentPrice = tuple of (price: Decimal, quoted_at: DateTime, obtained_at: DateTime, delayed: $\mathbb{B}$), a read-only copy of an asset's newest price. quoted_at is when the provider set the price, and obtained_at is when the server got it, the time FR33 asks to show with the price.

UnknownAssetError, a DomainError with code "unknown_asset", status 404 and the message "There is no asset with that symbol." Status 404 is HTTP's status for something that doesn't exist, so the phone can tell it apart from Trading's refusals (409).

### 13.3.2 Exported Constants

PLUGIN: Plugin = (name: "market", router: the two routes below, lifespan: lifespan, health: health). Both of Market's routes need no signed-in user. What they send is the same for everyone and is no user's data, so SEC-15 doesn't ask for one. They take now from get_clock of Clock (M2) and session from get_db of Database (M1).

### 13.3.3 Exported Access Programs

| Name | In | Out | Exceptions |
|---|---|---|---|
| get_asset_by_symbol | Session, String | AssetInfo | UnknownAssetError |
| get_asset | Session, ID | AssetInfo | UnknownAssetError |
| get_current_price | Session, ID, DateTime | CurrentPrice or None | - |
| is_delayed | DateTime, DateTime | $\mathbb{B}$ | - |
| is_market_open | DateTime | $\mathbb{B}$ | - |
| GET /v1/assets | - | tuple of (market_open: $\mathbb{B}$, assets: seq of tuple of (symbol: String, name: String, price: Money or None, price_currency: String, quoted_at: DateTime or None, obtained_at: DateTime or None, delayed: $\mathbb{B}$)) | - |
| GET /v1/market/usage | - | tuple of (calls_last_minute: $\mathbb{Z}$, busiest_minute_today: $\mathbb{Z}$, calls_today: $\mathbb{Z}$, limit_per_minute: $\mathbb{N}$) | - |

## 13.4 Semantics

### 13.4.1 State Variables

assets: set of tuple of (asset_id: ID, symbol: String, name: String, asset_type: String, price_currency: String, is_tradable: $\mathbb{B}$), every asset, with each symbol once.

prices: set of tuple of (asset_id: ID, price: Decimal, price_currency: String, quoted_at: DateTime, obtained_at: DateTime), every price saved, at most one per asset and quoted_at, so a quote the provider repeats while nothing trades is stored once. An asset's current price is the one with the latest quoted_at (FR32). Every price_currency, here and in assets, is QUOTE_CURRENCY from Constants (M7), and the database refuses any other. A Quote carries no currency, so prices stay in one currency because every asset is quoted in QUOTE_CURRENCY (FR29, SEC-27).

requests: seq of tuple of (requested_at: DateTime, symbol: String, outcome: String), one per call to the provider in a committed poll (see poll_once). GET /v1/market/usage counts them, so the team can check the calls stay within the provider's request limits (SC4).

poller: {off, running, stopped}, the state of the price poller's thread. It is off until lifespan

starts the thread and again once the server stops, running while the thread runs, and stopped if the thread ends while the server runs.

lastPollFailed: $\mathbb{B}$, whether the latest poll failed (see run_poller). It is false when lifespan starts the poller.

### 13.4.2 Environment Variables

MARKET_ALWAYS_OPEN: $\mathbb{B}$, default false, treats the market as always open, so the team can develop in the evening.
MARKET_POLLER_ENABLED: $\mathbb{B}$, default false, starts the price poller with the server. The default keeps tests, and any server not set up to poll, from calling the provider.

### 13.4.3 Assumptions

- The database migrations add the assets. Market never adds or removes one. Doing that without a new release (FR30) isn't built yet.
- At most one running server has MARKET_POLLER_ENABLED on, so prices are fetched once for all users, within the provider's request limits (SC4).
- Every time given to Market has a time zone.

### 13.4.4 Access Routine Semantics

get_asset_by_symbol(session, symbol):

- output: *out* := the AssetInfo of $a \in$ assets with $a$.symbol = symbol, even if not tradable
- exception: *exc* := (no $a \in$ assets has $a$.symbol = symbol $\Rightarrow$ UnknownAssetError)

get_asset(session, asset_id):

- output: *out* := the AssetInfo of $a \in$ assets with $a$.asset_id = asset_id
- exception: *exc* := (no $a \in$ assets has $a$.asset_id = asset_id $\Rightarrow$ UnknownAssetError)

get_current_price(session, asset_id, now):

- output: *out* := None if the asset has no price yet, else (price, quoted_at and obtained_at of its current price $p$, delayed: is_delayed($p$.quoted_at, now))
- exception: None

is_delayed(quoted_at, now):

- output: *out* := is_market_open(now) $\wedge$ now $-$ quoted_at $>$ limit, where limit is QUOTE_STALENESS_LIMIT from Constants (M7). It goes by quoted_at, since an old quote lags the real market (SEC-2) even if the server only just got it. While the market is closed, no price is delayed (FR33), since the last price then stays the same until the open.

- exception: None

is_market_open(now):

- output: *out* := MARKET_ALWAYS_OPEN $\vee$ is_trading_time(now). Polling, delayed prices and Trading (M12) all use it, so they agree on when the market is open.

- exception: None

GET /v1/assets():

- output: *out* := (market_open: is_market_open(now), assets: for each tradable $a \in$ assets by symbol, its symbol, name and price_currency, with the price (as Money), quoted_at, obtained_at and delayed from get_current_price, or None, None, None and false before its first price). Only tradable assets are listed, since a removed one may no longer be bought (FR30). market_open lets the phone tell the user the market is closed, so a refused buy is explained (HA H19).

- exception: None

GET /v1/market/usage():

- output: *out* := (calls_last_minute: $|R(\text{now} - 1 \text{ minute})|$, busiest_minute_today: the most entries of $R$(start) in one clock minute, or 0 if there are none, calls_today: $|R(\text{start})|$, limit_per_minute: FINNHUB_CALLS_PER_MINUTE of Market Providers (M3)), where $R(t)$ is the entries of requests from $t$ to now, and start is the midnight that begins now's New York day, the market's day

- exception: None

### 13.4.5 Local Functions

HOLIDAYS: set of Date, the weekdays in 2026 and 2027 that the New York Stock Exchange closes for a holiday.
EARLY_CLOSES: set of Date = {2026-11-27, 2026-12-24, 2027-11-26}, its early closes. Both are copied from the exchange's published calendar. After 2027, every weekday counts as a full trading day, so the next year's dates are added each December.

is_trading_time: DateTime $\to \mathbb{B}$
is_trading_time(now) := $d$ is a weekday not in HOLIDAYS $\wedge$ 9:30 $\leq t < c$, where $d$ and $t$ are now's day and time in New York (NEW_YORK, the America/New_York time zone), and $c$ is

13:00 if $d \in$ EARLY_CLOSES, else 16:00. These are the New York Stock Exchange's hours, which every US exchange shares, since every asset is a US stock. Reading now in New York time handles daylight saving time.

POLL_INTERVAL: Duration = 60 seconds, the wait after each check of should_poll. A poll makes one call per tradable asset. Polling once a minute stays within the provider's limit while it allows at least that many calls a minute (SC4).
POLL_AFTER_CLOSE: Duration = 15 minutes, how long polling goes on after the close, since the provider's closing price can arrive a few minutes late.
STOP_TIMEOUT: Duration = 15 seconds, how long the server waits for the poller to stop, so a poll under way can finish but a provider that doesn't answer can't keep the server from stopping.

should_poll: DateTime $\to \mathbb{B}$
should_poll(now) := is_market_open(now) $\vee$ is_market_open(now $-$ POLL_AFTER_CLOSE)

poll_once(session: Session, provider: MarketDataProvider, now: DateTime) $\to \mathbb{B}$: for each tradable asset, it gets a quote from provider.get_quote, saves it with save_quote and adds (now, symbol, outcome) to requests. outcome is "ok" for a quote. A RateLimitedError gives "rate_limited" and ends the poll, so the poller doesn't keep calling past the provider's request limits (SC4). An UnknownSymbolError gives "unknown_symbol" and any other MarketDataError gives "error". A price the database refuses is logged and undoes only its own save. It does not commit. It returns true only if it saved a quote for every tradable asset.

save_quote(session: Session, asset_id: ID, $q$: Quote) adds $q$ to prices with obtained_at the database's time. If the asset has a price at $q$.quoted_at already, an equal price is kept and a different one is replaced, with a new obtained_at, since the provider has corrected it.

run_poller(stop: $\mathbb{B}$, provider: MarketDataProvider): until stop is true, if should_poll(now) with now := system_clock(), it runs poll_once at now in a new SessionLocal session and commits. Either way, it then waits POLL_INTERVAL. A test can pass its own clock, session factory and interval instead (dependency injection). A poll that raises an error is logged and rolled back, so one bad minute never stops the thread. After each poll, lastPollFailed := (the poll raised an error, or poll_once returned false). So a rate limit, a provider that is down, a wrong key or a single symbol without a price all count as a failed poll.

lifespan: Lifespan. At start it reads the two environment variables and get_provider_settings of Market Providers (M3) and runs check_always_open, and a ValidationError or ValueError stops the server, so bad settings are found at once, even with the poller off. If MARKET_POLLER_ENABLED, run_poller runs on its own thread with the provider from build_provider until the server stops, which waits up to STOP_TIMEOUT. It sets poller and lastPollFailed as their descriptions say.

health: HealthCheck
health() := (poller = off $\Rightarrow$ ("ok", "poller off") | poller = stopped $\Rightarrow$ ("degraded", "poller stopped") | poller = running $\wedge$ lastPollFailed $\Rightarrow$ ("degraded", "last poll failed") | poller = running $\wedge\neg$lastPollFailed $\Rightarrow$ ("ok", None)). A failed poll counts as "degraded", since prices

may then stop updating while no request sees an error (HA H13). It shows until the next poll, which may be the next trading day. The poller being off is “ok”, since only one server polls.

check_always_open(market: tuple of (market_always_open: $\mathbb{B}$, market_poller_enabled: $\mathbb{B}$), providers: ProviderSettings) raises ValueError if market has MARKET_ALWAYS_OPEN on and providers.market_provider $\neq$ “fake”, so real prices are never shown as live while the market is closed.

# 14 MIS of Trading

## 14.1 Module

Trading (M12 in the MG)

## 14.2 Uses

Database (M1), Clock (M2), Users (M8), Wallet (M9), Market (M10), Plugins (M22), JSON Format (M23)

## 14.3 Syntax

### 14.3.1 Exported Types

Trading's errors are DomainErrors with HTTP status 409, each with its code and message:

- RequestIdReusedError, "request_id_reused", "This request ID was already used for a different trade."
- MarketClosedError, "market_closed", "The market is closed. Try again when it opens."
- PriceOutOfDateError, "price_out_of_date", "This stock's price is out of date, so it can't be bought right now."
- PriceChangedError, "price_changed", "The price is now $x$. Check it and confirm again.", with $x$ the new price to four decimals.

The local function buy says when each is raised. Status 409 says the request fits but conflicts with the market or an earlier trade, unlike Market's 404 for a stock that doesn't exist.

HoldingValue = tuple of (symbol: String, quantity: $\mathbb{Z}$, price: Money, quoted_at: DateTime, delayed: $\mathbb{B}$, value: Money), one holding in the reply to GET /v1/me/portfolio.

### 14.3.2 Exported Constants

PLUGIN: Plugin = (name: "trading", router: the two routes below, lifespan: None, health: None)

### 14.3.3 Exported Access Programs

| Name | In | Out | Exceptions |
|---|---|---|---|
| POST /v1/me/trades | ID, String, {buy}, ℕ, Money | tuple of (trade_id: ID, status: String, unit_price: Money, currency_amount: Money, executed_at: DateTime, cash_balance: Money) | RequestIdReusedError, MarketClosedError, UnknownAssetError, PriceOutOfDateError, PriceChangedError, InsufficientFundsError, NotAuthenticatedError |
| GET /v1/me/portfolio | - | tuple of (cash_balance: Money, holdings: seq of HoldingValue, holdings_value: Money, total_value: Money, simulated: 𝔹) | NotAuthenticatedError |

## 14.4 Semantics

### 14.4.1 State Variables

trades: set of Trade, every trade made, where Trade = tuple of (trade_id: ID, request_id: ID, portfolio_id: ID, asset_id: ID, trade_type: String, quantity: ℕ, unit_price: Decimal, currency_amount: Decimal, cash_balance_after: Decimal, status: String, balance_change_id: ID, created_at: DateTime). A request_id appears at most once among a portfolio's trades. It is unique per portfolio, so the portfolio's lock is enough to check it, and no user gets back another's trade. Every trade is a "buy" with status "executed", and the database refuses any other. Selling (FR36), queued trades (FR40) and refusing a trade before the user accepts the simulation notice (FR34) come at Revision 0. A trade keeps cash_balance_after, so a repeat gets the same reply. It also keeps the balance_change_id of the one payment for it, so balances can be checked against trades (HA H21).

holdings: set of tuple of (portfolio_id: ID, asset_id: ID, quantity: ℤ), the shares each portfolio holds, at most one entry per portfolio and stock, with quantity never below 0, which the database checks, so a sale at Revision 0 can't sell more shares than are held (FR37, HA H20).

When Wallet deletes a portfolio, its trades and holdings go with it.

### 14.4.2 Environment Variables

None

### 14.4.3 Assumptions

The lock from lock_live_portfolio lasts until the session's transaction ends, by a commit, a rollback or closing the session, so requests that change one portfolio run one at a time. Every stock in holdings has at least one price, since it was bought at one and no price is ever deleted.

### 14.4.4 Access Routine Semantics

Both routes take session from get_db, now from get_clock and $u :=$ get_current_user().

POST /v1/me/trades(request_id, symbol, trade_type, quantity, expected_price):

- transition: the route locks the portfolio with locked := lock_live_portfolio(session, $u$.user_id), runs $(t, new) :=$ buy(session, locked, request, now) on the request body, and commits once. So the payment, the shares and the trade are saved together or not at all (FR37). With the lock first, two copies of one request run one after the other, and if the first made a trade, the second finds it (FR38).

- output: the status is 201 for a new trade, HTTP's status for something made, and 200 for a repeat, which makes nothing. $out :=$ ($t$.trade_id, $t$.status, $t$.unit_price, $t$.currency_amount, $t$.created_at in UTC, $t$.cash_balance_after), so a repeated request_id gets the same body as the first answer, and a phone that lost that answer learns its trade went through (HA H46). UTC keeps the body the same, since a stored time may come back in another time zone.

- exception: $exc :=$ the NotAuthenticatedError of get_current_user, with status 401, or else the exception buy raises, with status 404 for UnknownAssetError and 409 for the rest. A refused request saves nothing, so sending it again runs every check again. A trade_type other than "buy" doesn't fit the body yet, so it gets JSON Format's 422.

GET /v1/me/portfolio():

- output: $out :=$ portfolio_value(session, live_portfolio_id(session, $u$.user_id), now) with simulated = true, so the phone labels the values as simulated (FR43). It doesn't lock, so showing the portfolio never waits for a buy. A buy that commits while it runs can then make one reply off by that buy, and the next is right.

- exception: $exc :=$ the NotAuthenticatedError of get_current_user, with status 401

### 14.4.5 Local Functions

TradeRequest = tuple of (request_id: ID, symbol: String, trade_type: {buy}, quantity: $\mathbb{N}$, expected_price: Money), the body of POST /v1/me/trades. request_id is new for each buy the user confirms and the same for each retry of it (an idempotency key). expected_price is the price the user confirmed, so the buy runs only at that price (SEC-29).

buy(session: Session, locked: LockedPortfolio, request: TradeRequest, now: DateTime) → tuple of (t: Trade, new: $\mathbb{B}$) holds the trade rules and never commits, by the commit rule of Database (M1). Here $p :=$ locked.portfolio_id, and old is the trade in trades with portfolio_id $p$ and request_id request.request_id, or None. old matches request when it has the same symbol and quantity, and old.unit_price = request.expected_price. For a new request (old = None), $a :=$ get_asset_by_symbol(session, request.symbol), $c :=$ get_current_price(session, $a$.asset_id, now) and cost := request.quantity $\times$ $c$.price, exact, since it is a whole number times a price (NFR1). The one read $c$ gives the price checked, the cost and the trade's price, so they can't differ.

- transition: for a new request that passes every check, $b :=$ change_balance(session, locked, −cost, trade_purchase) pays for it, $p$'s holding of $a$ gains request.quantity shares (from 0 if none), and trades := trades $\cup\{t\}$, where $t :=$ (a new ID, request.request_id, $p$, $a$.asset_id, "buy", request.quantity, $c$.price, cost, $b$.cash_balance_after, "executed", $b$.balance_change_id, now). It takes now, not the database's time, so the trade's time is the one its market hours and price were checked at.

- output: *out* := (old matches request ⇒ (old, false) | old = None ⇒ ($t$, true)), so a repeat gets its first trade back, even after the market closes or the price moves, since that trade already went through.

- exception: for a repeat, *exc* := (old ≠ None ∧ old does not match request ⇒ RequestIdReusedError).
  A different trade with a used ID is refused, since it is not a retry, and buying it too would give the ID two trades.
  For a new request the first of these that holds is raised, in this order:
  *exc* := (¬is_market_open(now) ⇒ MarketClosedError
  | no asset has symbol request.symbol ⇒ UnknownAssetError
  | $c$ = None ∨ $c$.delayed ⇒ PriceOutOfDateError
  | $c$.price ≠ request.expected_price ⇒ PriceChangedError
  | get_balance(session, $p$) < cost ⇒ InsufficientFundsError)
  MarketClosedError refuses a buy while the market is closed, so it never runs at an old price (HA H19). Queueing it instead (FR40) comes at Revision 0. PriceOutOfDateError refuses a delayed price (FR39), or no price yet, since either way there is no current price to buy at. buy doesn't check is_tradable yet, since removing a stock (FR30) isn't built. PriceChangedError's message gives the new price, $c$.price, so the phone can show it and ask the user to confirm again (SEC-29). InsufficientFundsError comes from change_balance (FR37).

portfolio_value(session: Session, portfolio_id: ID, now: DateTime) → tuple of (cash_balance: Decimal, holdings: seq of HoldingValue, holdings_value: Decimal, total_value: Decimal) only reads.

- output: *out* := $(b, H, s, b+s)$, where $b :=$ get_balance(session, portfolio_id), $H$ has one entry per holding $h$ of portfolio_id, in order of symbol, so every reply lists them in the

same order, and $s$, the sum of their values, is the portfolio value of FR42, which leaves the cash out. With $c :=$ get_current_price(session, $h$.asset_id, now), $h$'s entry is (its symbol, $h$.quantity, $c$.price, $c$.quoted_at, $c$.delayed, $h$.quantity $\times$ $c$.price). A delayed price is still used, since FR42 asks for each stock's most recent price, and delayed flags it, so the phone marks it (FR33). A holding gives quoted_at, the time delayed is judged by, but not yet obtained_at, the time FR33 asks to show. No amount needs rounding, since none comes from a division.

- exception: None

# 15 MIS of Plugins

## 15.1 Module

Plugins (M22 in the MG)

## 15.2 Uses

None

## 15.3 Syntax

### 15.3.1 Exported Types

Plugin = tuple of (name: String, router: Router or None, lifespan: Lifespan or None, health: HealthCheck or None), a record of what one module adds to the server. create_app of Control (M6) depends on this record alone, never on a module (the plug-in pattern). It cannot be changed once built, so the check in new Plugin still holds when Control reads it.

- name: the module's folder name, such as "market".
- router: the module's routes, which Control (M6) adds under /v1.
- lifespan: the module's background work, which runs while the server runs.
- health: the module's health check, which Control (M6) shows in GET /v1/health.

Lifespan = App $\rightarrow$ work that starts with the server and stops when it shuts down, such as the price poller of Market (M10). A Lifespan stops its work in a finally block, so the work still stops if another module fails to start.

HealthCheck = () $\rightarrow$ ModuleHealth, a check of the module's own state.

ModuleHealth = tuple of (status: String, detail: String or None), a module's health. Its status is "ok" or "degraded", and its detail is a short fixed text for the team, such as "poller stopped".

Router: a group of HTTP routes (a FastAPI APIRouter).

App: the web app that serves the routes (a FastAPI app).

### 15.3.2 Exported Constants

None

### 15.3.3 Exported Access Programs

| Name | In | Out | Exceptions |
|---|---|---|---|
| new Plugin | String, Router or None, Lifespan or None, HealthCheck or None | Plugin | ValueError |

## 15.4 Semantics

### 15.4.1 State Variables

None

### 15.4.2 Environment Variables

None

### 15.4.3 Assumptions

- Every Lifespan follows the start and stop rule in its type above.
- A HealthCheck reads only its module's own state in memory, with no database or network call, so it answers at once. Its detail never holds a secret or an error's own text, because GET /v1/health needs no signed-in user.

### 15.4.4 Access Routine Semantics

new Plugin(name, router, lifespan, health):

- output: $out$ := (name: name, router: router, lifespan: lifespan, health: health). Each of router, lifespan and health is None when the module leaves it out.
- exception: $exc$ := (lifespan $\neq$ None $\wedge$ health = None $\Rightarrow$ ValueError). A module with background work must report its health, because that work can fail without any request seeing it.

### 15.4.5 Local Functions

None

# 16 MIS of JSON Format

## 16.1 Module

JSON Format (M23 in the MG)

## 16.2 Uses

None

## 16.3 Syntax

### 16.3.1 Exported Types

Money = Decimal, an amount, such as a price. It is sent to the phone as a JSON string with exactly four decimals (see _to_text), not as a JSON number, since the phone would read a number as a float, which can't hold most decimals exactly. Four decimals keep any rounding far below VALUE_TOLERANCE (NFR1).

DomainError = tuple of (code: String, message: String, status: $\mathbb{N}$), an error a route raises to refuse a request it understood. The code names the kind of refusal, the message says why in words a user can read, and the status is the HTTP status of the reply. The defaults are code "domain_error", message "This request could not be completed." and status 400. Each kind of refusal is a subtype of DomainError with its own code, and it can also set its own message and status.

Refusal = tuple of (detail: tuple of (code: String, message: String)), the JSON body handle_domain_error sends, with HTTP status $e$.status, for every DomainError $e$ a route raises, so the phone handles every module's refusals the same way:

```
{"detail": {"code": e.code, "message": e.message}}
```

A request that doesn't fit a route's types, such as an amount with over 4 decimals, never reaches the route. FastAPI raises a RequestValidationError, its own type, which holds a FieldError for each field that doesn't fit. A FieldError, in FastAPI's shape, says what is wrong (type and msg, plus ctx for some) and where (loc). FastAPI also keeps the value sent with each one (input), which handle_validation_error leaves out.

### 16.3.2 Exported Constants

None

### 16.3.3 Exported Access Programs

| Name | In | Out | Exceptions |
|---|---|---|---|
| new Money | Decimal or $\mathbb{Z}$ or $\mathbb{R}$ or String | Money | ValidationError |
| new DomainError | String or None | DomainError | - |
| handle_domain_error | DomainError | tuple of (status: $\mathbb{N}$, body: Refusal) | - |
| handle_validation_error | RequestValidationError | tuple of (status: $\mathbb{N}$, body: tuple of (detail: seq of FieldError, each without its input)) | - |

## 16.4 Semantics

### 16.4.1 State Variables

None

### 16.4.2 Environment Variables

None

### 16.4.3 Assumptions

Every amount a route sends to the phone has the type Money. An amount that comes from a division or an average is rounded to four decimals before it goes in a Money field.

### 16.4.4 Access Routine Semantics

new Money(amount):

- output: *out* := the Decimal that amount names
- exception: *exc* := (amount is not a finite number, or cannot be written exactly with four decimals ⇒ ValidationError). It is refused, never rounded, so no amount, such as the price a user confirmed, is changed quietly.

new DomainError(message):

- output: *out* := (code: $k$.code, message: (message $\neq$ None ⇒ message | message = None ⇒ $k$.message), status: $k$.status), where $k$ is the kind of DomainError being made and message is None when left out. A refusal is given a message when it needs details, such as PriceChangedError of Trading (M12), which gives the new price (SEC-29).
- exception: None

handle_domain_error($e$):

- output: *out* := (status: $e$.status, body: the Refusal of $e$)
- exception: None

handle_validation_error($e$):

- output: *out* := (status: 422, body: (detail: the FieldErrors of $e$, each without its input)), the reply FastAPI's own handler sends, but without the values the phone sent. Some can't be written as JSON, such as NaN in a body, and sending one back would turn the 422 (SEC-25) into a 500.
- exception: None

### 16.4.5 Local Functions

_to_text: Money → String
_to_text($m$) := $m$ with exactly four digits after the decimal point, such as "84.0000" for 84. A zero is always "0.0000", never "-0.0000", since a minus sign marks a loss (FR44).

# References

Carlo Ghezzi, Mehdi Jazayeri, and Dino Mandrioli. *Fundamentals of Software Engineering*. Prentice Hall, Upper Saddle River, NJ, USA, 2nd edition, 2003.

Daniel M. Hoffman and Paul A. Strooper. *Software Design, Automated Testing, and Maintenance: A Practical Approach*. International Thomson Computer Press, New York, NY, USA, 1995. URL http://citeseer.ist.psu.edu/428727.html.

# 17 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

## 17.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

## 17.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

## 17.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

# 18 Appendix — Reflection

[Not required for CAS 741 projects —SS]

The information in this section will be used to evaluate the team members on the graduate attribute of Problem Analysis and Design.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. Which of your design decisions stemmed from speaking to your client(s) or a proxy (e.g. your peers, stakeholders, potential users)? For those that were not, why, and where did they come from?

4. While creating the design doc, what parts of your other documents (e.g. requirements, hazard analysis, etc), it any, needed to be changed, and why?

5. What are the limitations of your solution? Put another way, given unlimited resources, what could you do to make the project better? (LO_ProbSolutions)

6. Give a brief overview of other design solutions you considered. What are the benefits and tradeoffs of those other designs compared with the chosen design? From all the potential options, why did you select the documented design? (LO_Explores)

7. (After you have implemented another team's module, which means this isn't filled in until after the original deadline). What did you learn by implementing another team's module? Were all the details you needed in the documentation, or did you need to make assumptions, or ask the other team questions? If your team also had another team implement one of your modules, what was this experience like? Are there things in your documentation you could have changed to make the process go more smoothly for when an "outsider" completes some of the implementation?

Your reflection should be about concepts learned in your SE education, such as modular decomposition, cohesion, coupling, information hiding, etc. You should critique the quality and utility of the specification you received.

You should create a PR for your implemented module. A link to the PR should be part of your reflection.